

Lecture 9 Sorting(I), Sorting (II)

Sorting

The problem of sorting an array of elements.

- Assumption: Contains only **integers**.
- Internal sorting: The number of elements is relatively **small** and sorting **can be done in main memory**
Insertion sort, Shell sort, heap sort, merge sort, quick sort, bucket sort and Radix sort
- External sorting: The number of elements is too large to sort once.

Preliminaries

```
void X_Sort (ElementType A[], int N)
```

where X is different types of sorting. Here consider only **internal sorting**.

Insertion Sort

Repeatedly insert a new element into an **already sorted array**.

- The array is divided into sorted part and unsorted part
- Choose the first element in the unsorted part
- For each new element, search backward through sorted elements
- Move elements until proper position is found
- Place element in proper position
- This procedure is called a **pass** of sorting.

Totally $N - 1$ passes. For the k -th pass, the elements in position 0 to $k - 1$ is sorted.

The time complexity is $O(N^2)$ for worst case(reversely sorted), $O(N)$ for best case(sorted).

An **inversion** in an array of number is any ordered pair (i, j) having the property that $i < j$ but $A[i] > A[j]$. For each pass of insertion sort, it can only remove one inversion. So $O(I + N)$ for average case, where I is the number of inversion.

The theorem includes:

- The average number of inversions in an array of N distinct numbers is $N(N - 1)/4$.
- Any algorithm that exchange adjacent elements require $\Omega(N^2)$.

Thus $O(N^2)$ for average cases.

Shell Sort

The idea of shell sort

Move elements a longer distance each time. (Optimization of insertion sort)

- Divide the array to be sorted into subarrays every five elements (5-sort, 5 is called the gap).
- Insertion sort for each subarrays, then use 3-sort into subarrays and repeat the operations until finishing 1-sort.

Example:

初始	81	94	11	96	12	35	17	95	28	58	41	75	15
在 5-排序后	35	17	11	28	12	41	75	15	96	58	81	94	95
在 3-排序后	28	12	11	35	15	41	58	17	94	75	81	96	95
在 1-排序后	11	12	15	17	28	35	41	58	75	81	94	95	96

For this example, the first division of subarrays are $[81, 35, 41]$, $[94, 17, 75]$, $[11, 95, 15]$, $[96, 28]$, $[12, 58]$ and sort for each subarray on the base of original array. Then choose 3 as gap based on 5-sort, choose 1 as gap based on 3-sort and finish the sort.

Diminishing gaps

For a large array, we do not use 5-sort. We use N -sort, where N depends on the size of array. N is called the gap size or interval size.

We want to do several stages to reduce the gap size each time until to 1.

So

- define an increment sequence $h_1 < h_2 < \dots < h_{t-1} < h_t$ where $h_1 = 1$.
- Do h_i -sort then h_{i-1} -sort and so on until 1-sort.
- Each sort based on former sort.

Shell's increment sequence:

$$h_t = \lfloor N/2 \rfloor, h_k = \lfloor h_{k+1}/2 \rfloor$$

But it is not good enough due to every subarray is the subset of former subarray so lots of repeated sorting.

Hibbard's Increment sequence:

$$h_k = 2^k - 1$$

For Hibbard's increment, the worst case $T(N) = O(N^{3/2})$

Selection Sort

- Find the smallest in the array and exchange it with the element in the first position
- Find the second smallest element to the second position
- Continue until the last element.

For all cases the time complexity is $O(N^2)$.

Bubble Sort

- Move from the front to the end.
- Put the largest value to the end using pair-wise comparisons and swapping.
- Repeat the steps until the first two elements or there is no swapping.

Time complexity $O(N)$ for best case, $O(N^2)$ for worst case, $O(N^2)$ for average case.

Heap Sort

Algorithm 1 (not good): Using extra array to express min heap, delete one element in the heap until the heap is empty.

$$T(N) = O(N \log N), S(N) = O(N)$$

Algorithm 2: Use max heap, each step delete an element, and save the deleted max element in the last position of the current heap.

$$T(N) = O(N \log N)$$

Merge Sort

Often use in external sort.

Merge two sorted list into a larger sorted list.

- Use two input arrays A, B and an output array C and three iterators Aptr, Bptr, Cptr which are settled at the start of corresponding array.
- Copy the smaller in A[Aptr] and B[Bptr] into corresponding position in C and move corresponding iterator to the next.
- Repeat the former steps until one array is traversed. Then copy the rest elements of another array into C.

$$T(N) = O(N), S(N) = O(N).$$

Merge sort an array

To Mergesort an array recursively:

- Divide the array into two halves
- Mergesort the left half
- Mergesort the right half
- Merge two sorted halves.

Base case is only one element to be sorted.

We wish the `tmparray` as a argument of function($O(N)$) rather than create temporary in the function($O(N \log N)$) to reduce the space complexity.

Time complexity is $O(N \log N)$.

Iterative version: Merge two adjacent element into a sorted list.

Quick Sort

- Choose some element called a **pivot**(枢轴).
- Perform a sequence of exchanges so that
 - All elements that less than this pivot are to its left.
 - All elements that greater than this pivot are to its right.
- The pivot is placed at the right place **once and for all**.
- Divides the sublist into two smaller sublist.
- Each of which may then be sorted independently in the same way.

$$T(N) = T(i) + T(N - i - 1) + cN$$

where i is the number of elements in one side.

For best case, the number of elements of both sides of the pivot is equivalent. ($O(N \log N)$)

For worst case, $O(N^2)$

For average case, $O(N \log N)$.

Partitioning Strategy

- Choose a pivot(last element, first element, mid element or random)
- **Traverse by two pointers**
 - Left pointer traverse from left to right to find the element **bigger or equal** than pivot.
 - Right pointer traverse from right to left to find the element **smaller or equal** than pivot.
 - Swap the element pointed by the two pointers **when both be found**. Repeat the steps until the left pointer coincide with the right pointer.
 - Swap the pivot with the element pointed by left pointer.

The reason to exchange the key equivalent to pivot is to avoid **all equivalent elements are set in one side** of pivot, causing unbalance of partition.

Picking the Pivot

A terrible way is to choose the first or last element directly. If the array is sorted it will be the worst case.

A safe maneuver is to randomly select but it is expensive.

So most common method is to use median-of-three partitioning. That is, `pivot = median(left, center, right)`.

Small Arrays

For quick sort, the base case is the element only one.

But for small arrays with the number of elements less than 20, quick sort is slower than insertion sort. (The constant factor of insertion sort is smaller)

Thus actually, we need not to recurse to the base case. We define a cutoff when N gets small (e.g. 10) and use other efficient algorithms such as insertion sort.